
Plataforma de Producción Académica Institucional

Documentación técnica del proyecto final

CURSO

Programación Web Avanzada — SOFT-12, sección SCP0

UNIVERSIDAD

CENFOTEC — Escuela de Ingeniería del Software

DOCENTE FACILITADOR

Álvaro Cordero Peña

PERIODO

2026-C2

AUTOR

_____ (trabajo individual)

STACK

React 18 · Node.js + Express · MySQL 8 en Docker

CONTENIDO

1. Arquitectura general
2. Modelo de datos (MER)
3. Control de acceso basado en roles (RBAC)
4. Consulta con volumen de datos
5. Integración con APIs externas
6. Despliegue

Arquitectura general

El sistema se divide en tres capas independientes que se comunican por HTTP. El navegador nunca accede directamente a la base de datos ni conoce las claves de los servicios externos: todo pasa por el API.

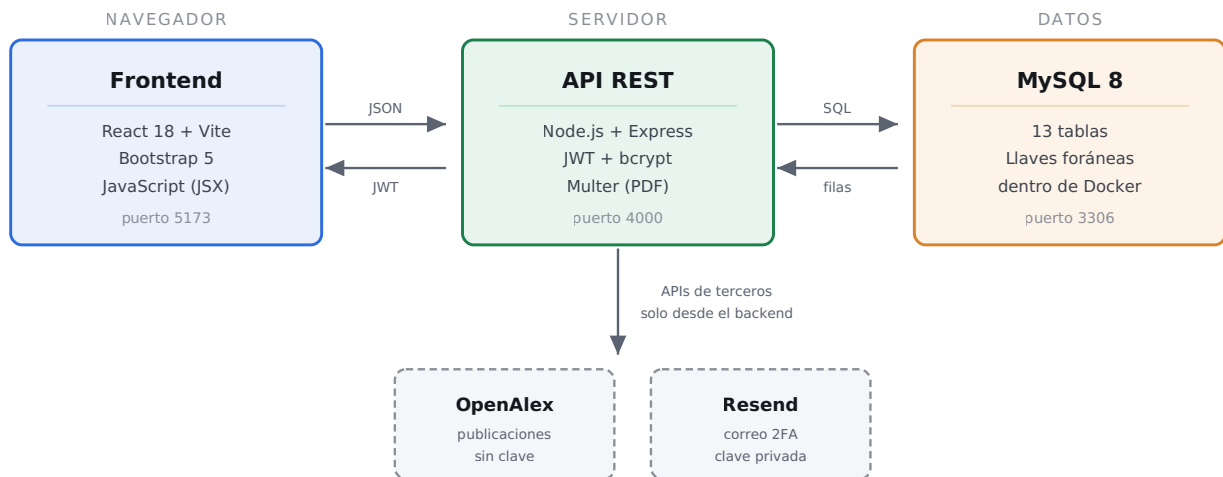


Figura 1 — Las tres capas del sistema y las APIs externas

DECISIÓN DE SEGURIDAD

Las claves de las APIs de terceros y las credenciales de MySQL viven únicamente en el backend, dentro del archivo `.env`, que no se sube al repositorio. El frontend solo conoce la dirección del API propio.

Docker y MySQL

Docker no reemplaza a MySQL: es la herramienta que permite ejecutarlo sin instalarlo en el sistema operativo. Es el mismo enfoque del laboratorio del curso, donde se levantó el motor de base de datos con un solo comando.

```
# Levanta MySQL 8 en un contenedor
docker run --name mysql-produccion \
  -e MYSQL_ROOT_PASSWORD=root \
  -e MYSQL_DATABASE=produccion_academica \
  -p 3306:3306 \
  -d mysql:8.0
```

PARÁMETRO	FUNCIÓN
<code>--name</code>	Nombre fijo del contenedor, para reiniciarlo después con <code>docker start</code>
<code>MYSQL_ROOT_PASSWORD</code>	Contraseña del usuario <code>root</code> ; debe coincidir con el <code>.env</code>
<code>MYSQL_DATABASE</code>	Crea la base de datos vacía al arrancar
<code>-p 3306:3306</code>	Publica el puerto para que Node pueda conectarse
<code>-d</code>	Ejecuta en segundo plano y libera la terminal

Modelo de datos (MER)

Motor: **MySQL 8**, con motor de almacenamiento InnoDB para soportar llaves foráneas. El script completo está en [backend/src/db/schema.sql](#).

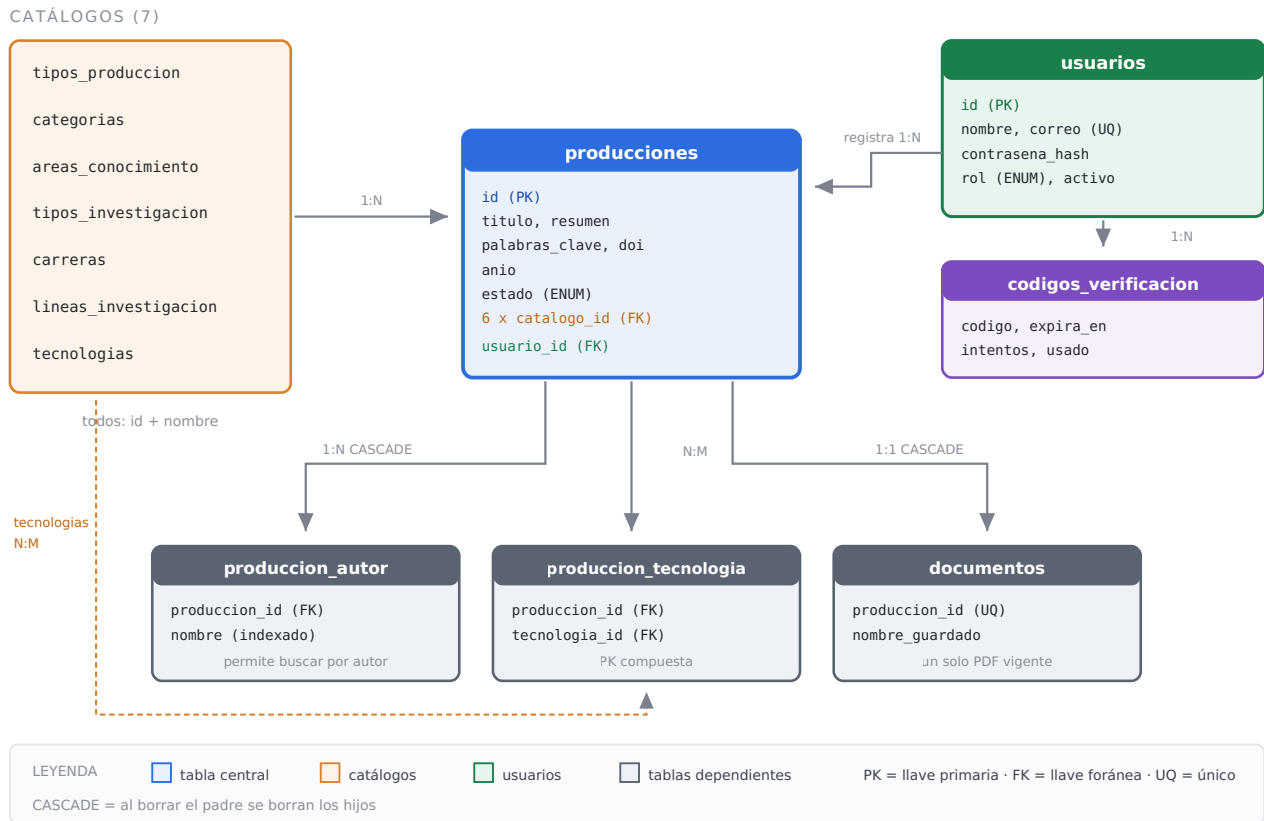


Figura 2 — Modelo entidad-relación (13 tablas)

Inventario de tablas

#	TABLA	FUNCIÓN
1	<code>usuarios</code>	Cuentas del sistema, con su rol y estado
2	<code>codigos_verificacion</code>	Códigos de 6 dígitos del inicio de sesión en dos pasos
3	<code>tipos_produccion</code>	Catálogo: tesis, artículo, ponencia...
4	<code>categorias</code>	Catálogo: pregrado, posgrado...
5	<code>areas_conocimiento</code>	Catálogo de áreas de conocimiento
6	<code>tecnologias</code>	Catálogo de tecnologías
7	<code>tipos_investigacion</code>	Catálogo: exploratoria, aplicada...
8	<code>carreras</code>	Catálogo de carreras o programas
9	<code>lineas_investigacion</code>	Catálogo de líneas de investigación
10	<code>producciones</code>	Tabla central del sistema
11	<code>produccion_autor</code>	Autores de cada producción (1:N)
12	<code>produccion_tecnologia</code>	Tecnologías de cada producción (N:M)
13	<code>documentos</code>	El archivo PDF adjunto (1:1)

Justificación del diseño

Los autores van en su propia tabla

La consigna pide buscar por autor. Si los autores estuvieran en una sola columna de texto (`"Karla Solano, Marco Jiménez"`), buscar obligaría a comparar dentro de ese texto y no se podría indexar bien. Con `produccion_autor` la búsqueda usa un índice sobre `nombre` .

Las tecnologías van en una tabla de relación

Una producción usa varias tecnologías y una tecnología aparece en varias producciones: es una relación de muchos a muchos, y eso se resuelve con una tabla intermedia con llave primaria compuesta.

`producciones.usuario_id` guarda quién registró cada fila

Sin ese dato no se podría aplicar la regla «*editar solo la producción propia*» del control de acceso, que distingue entre editar cualquier registro y editar únicamente los propios.

`documentos.produccion_id` es UNIQUE

Cada producción tiene un solo PDF vigente. Además de ser la regla del negocio, esta restricción permite unir la tabla en el listado sin que se dupliquen las filas del resultado.

El estado es un ENUM

Los tres estados (`borrador` , `revision` , `publicado`) son parte del flujo del sistema, no datos que el usuario administre, así que se validan en el motor y no en una tabla de catálogo.

Comportamiento del borrado

RELACIÓN	REGLA	MOTIVO
<code>produccion_autor</code> <code>produccion_tecnologia</code> <code>documentos</code>	<code>ON DELETE CASCADE</code>	No tienen sentido sin su producción
Catálogos opcionales	<code>ON DELETE SET NULL</code>	Borrar un área no debe borrar las producciones
<code>tipo_id</code> (obligatorio)	MySQL lo impide	El API responde <code>409</code> indicando que el registro está en uso
<code>usuarios</code>	Baja lógica (<code>activo = 0</code>)	Son dueños de producciones; borrarlos dejaría registros sin responsable

Control de acceso basado en roles

La consigna aclara que no basta con almacenar el rol: cada rol debe tener permisos diferenciados, aplicados tanto en el backend como en el frontend. El sistema define **4 roles** y **11 permisos**.

Matriz de permisos

PERMISO	ADMIN	COORDINADOR	DOCENTE	ESTUDIANTE
<code>usuarios.gestionar</code>	Sí	—	—	—
<code>catalogos.gestionar</code>	Sí	Sí	—	—
<code>produccion.consultar</code>	Sí	Sí	Sí	Sí
<code>produccion.crear</code>	Sí	Sí	Sí	Sí
<code>produccion.editar.propia</code>	Sí	Sí	Sí	Sí
<code>produccion.editar.cualquiera</code>	Sí	Sí	—	—
<code>produccion.eliminar</code>	Sí	Sí	—	—
<code>documento.subir</code>	Sí	Sí	Sí	Sí
<code>documento.descargar</code>	Sí	Sí	Sí	Sí
<code>dashboard.ver</code>	Sí	Sí	Sí	—
<code>externa.importar</code>	Sí	Sí	Sí	—

Esta matriz reproduce la de la consigna. Los tres permisos que la consigna no listaba en su tabla resumen se justifican así:

- `documento.subir` y `documento.descargar` se otorgan a los cuatro roles, porque cualquiera debe poder adjuntar el PDF de su propia producción y consultar los documentos.
- `externa.importar` no se otorga al estudiante: importar agrega registros al repositorio institucional a partir de fuentes externas, lo cual es una tarea de revisión académica. Lo importado además entra en estado `revision`.

Las tres capas de aplicación



Figura 3 — Las tres capas donde se aplica el control de acceso

PRINCIPIO CLAVE

Ocultar un botón **no protege nada**: cualquiera podría llamar al API directamente con `curl` sin pasar por la interfaz. La decisión autoritativa la toma siempre el backend; el frontend solo evita mostrar acciones que van a fallar.

Implementación en el backend

Cada ruta declara el permiso que necesita. El middleware lo verifica antes de ejecutar cualquier lógica:

```
router.delete("/:id", requierePermiso("produccion.eliminar"), ...)
```

El token JWT solo transporta el `id` y el `rol`. Los permisos se resuelven en el momento de atender la petición, y el usuario se vuelve a leer de la base de datos para que una cuenta desactivada no siga entrando con un token emitido antes.

La regla de propiedad

El permiso por sí solo no alcanza: hay que saber de quién es el registro. La función `puedeEditar()` resuelve esa distinción:

```
if (tienePermiso(usuario.rol, "produccion.editar.cualquiera")) return true;
return tienePermiso(usuario.rol, "produccion.editar.propia") && usuario.id === dueñoId;
```

Verificación con pruebas automatizadas

El comportamiento está comprobado por HTTP con Supertest, no solo por inspección del código:

CASO PROBADO	RESPUESTA ESPERADA
Estudiante consulta producciones	200
Estudiante entra al dashboard	403
Estudiante elimina una producción	403
Docente edita producción ajena	403
Docente edita su propia producción	200
Docente administra catálogos	403
Coordinador administra catálogos	200
Coordinador entra a gestión de usuarios	403
Petición sin token	401
Petición con token inválido	401

Además, la pantalla **Usuarios → Matriz de permisos** construye esta tabla leyéndola del backend, de modo que refleja exactamente lo que el servidor aplica.

Consulta con volumen de datos

La consigna pide que la consulta esté pensada para volúmenes grandes. Se resolvió con cinco medidas concretas.



Los autores y las tecnologías de la página se cargan en 2 consultas adicionales para todo el lote, no una por cada producción (se evita el problema N+1)

Figura 4 — Recorrido de una consulta con filtros y paginación

1. Paginación del lado del servidor

El API recibe `pagina` y `porPagina`, y usa `LIMIT` y `OFFSET`. Nunca se envían todos los registros al navegador. Se realiza una segunda consulta con `COUNT(*)` para conocer el total y calcular la cantidad de páginas.

2. Búsqueda de texto

La consigna permite «full-text o, como mínimo, búsqueda parcial sobre título, resumen y palabras clave». Se implementó la búsqueda parcial con `LIKE`, que es la opción mínima aceptada y la más simple de mantener.

3. Ordenamiento con lista blanca

Se puede ordenar por año, título o fecha de registro. El nombre de la columna no admite un marcador `?` dentro de un `ORDER BY`, así que se valida contra una lista fija. Si el valor del usuario se insertara directamente, sería una vía de inyección SQL.

```
const ORDENES = {
  año: "p.año",
  título: "p.título",
  creado_en: "p.creado_en",
};

const orden = ORDENES[req.query.ordenarPor] || "p.creado_en";
```

4. Filtros combinados en una sola consulta

Todas las condiciones se acumulan en un mismo `WHERE`, de modo que buscar texto y filtrar por año y por área no son tres consultas, sino una.

5. Autores y tecnologías sin consultas repetidas

Se cargan con dos consultas para toda la página de resultados, no una por cada producción, evitando el problema conocido como N+1.

PREVENCIÓN DE INYECCIÓN SQL

Todas las consultas usan marcadores `?` con parámetros; nunca se concatena el valor recibido del usuario. La biblioteca `mysql2` envía el SQL y los valores por separado, así que MySQL trata el valor como dato y jamás como instrucción. La única excepción es el nombre de columna del `ORDER BY`, resuelta con la lista blanca descrita arriba.

Integración con APIs externas

El sistema consume dos servicios de terceros. Ambos se consultan exclusivamente desde el backend.

SERVICIO	FUNCIÓN	AUTENTICACIÓN
OpenAlex	Catálogo público de publicaciones científicas	No requiere clave; solo un correo de contacto opcional
Resend	Envío del código de verificación por correo	Clave privada, almacenada solo en el backend

OpenAlex

Archivo: [backend/src/services/openalexService.js](#)

La consulta se realiza desde el backend y la respuesta se normaliza a la estructura propia del proyecto, de modo que el frontend no depende del formato de OpenAlex. Si en el futuro se cambiara de proveedor, el frontend no se vería afectado.

El detalle más complejo de la integración es que OpenAlex no entrega el resumen como texto, sino como un **índice invertido**: un objeto donde cada palabra apunta a las posiciones en que aparece. Hay que recorrerlo y reconstruir la frase.

```
// OpenAlex devuelve: { "Un": [0], "estudio": [1], "sobre": [2] }  
// Se reconstruye a: "Un estudio sobre"  
function rearmarResumen(indice) {  
  if (!indice) return null;  
  const palabras = [];  
  for (const palabra in indice) {  
    for (const posicion of indice[palabra]) palabras[posicion] = palabra;  
  }  
  return palabras.filter(Boolean).join(" ").trim();  
}
```

La importación aplica dos reglas de negocio: si el DOI ya existe en la base se responde [409](#) para evitar duplicados, y todo registro importado entra en estado [revision](#), nunca directamente como publicado, porque proviene de una fuente externa y requiere validación humana.

VERIFICACIÓN DE ESTA INTEGRACIÓN

La normalización de OpenAlex está cubierta por pruebas con respuestas simuladas, que validan la reconstrucción del índice invertido y el manejo de campos ausentes. Al simular la respuesta, las pruebas no dependen de la red ni de la disponibilidad del servicio externo.

Resend y la verificación en dos pasos

Archivo: [backend/src/services/correoService.js](#)



El token NO se entrega en el paso 1: sin el código correcto no hay acceso.

El código vence a los 10 minutos y admite 3 intentos.

Si la clave de Resend no está configurada, el código se imprime en la consola del backend.

Figura 5 — Inicio de sesión con verificación en dos pasos

VARIABLE CORREO_PRUEBAS

La cuenta gratuita de Resend, mientras use el remitente de prueba, solo entrega correos a la dirección con la que se creó la cuenta. Como el sistema tiene cuatro usuarios con correos distintos, la variable `CORREO_PRUEBAS` redirige todos los códigos a una sola bandeja e indica en el asunto a qué cuenta corresponde cada uno. En producción esta variable se deja vacía y cada usuario recibe su propio correo.

Despliegue

El proyecto se ejecuta completo en local mediante Docker, que es lo necesario para demostrar su funcionamiento.

Para publicarlo, siguiendo el esquema del laboratorio del curso, serían tres piezas independientes:

PIEZA	PLATAFORMA SUGERIDA	OBSERVACIÓN
Frontend	Vercel (sitio estático)	Se publica el resultado de <code>npm run build</code>
Backend	Render	Servicio Node en ejecución continua
Base de datos	Railway, Aiven o Clever Cloud	Render ofrece PostgreSQL gratuito, no MySQL

Cambios necesarios en producción

- **Backend:** `DB_HOST` , `DB_PORT` , `DB_USER` , `DB_PASSWORD` apuntando al proveedor de MySQL, y un `JWT_SECRETO` distinto al de desarrollo.
- **Frontend:** apuntar las llamadas al dominio del backend en lugar del proxy de Vite.

Verificación automatizada

El repositorio incluye un flujo de trabajo de GitHub Actions que, en cada `push` , levanta un contenedor temporal de MySQL, ejecuta el analizador de código y las pruebas automatizadas.

CONJUNTO	CANTIDAD	ALCANCE
Backend — unitarias	17	Matriz de permisos y normalización de OpenAlex
Backend — endpoints	38	Autenticación, CRUD, RBAC por HTTP, documentos
Frontend	12	Lógica de permisos y componente de paginación
Total	67	

Limitaciones conocidas

LIMITACIÓN	DESCRIPCIÓN Y CONSECUENCIA
Almacenamiento de documentos	Los archivos PDF se guardan en el disco del servidor. En los planes gratuitos de las plataformas de despliegue el disco se borra en cada actualización, por lo que los documentos subidos no sobrevivirían. Para producción real habría que usar un servicio de almacenamiento externo, como menciona la consigna.
Motor de base de datos	La consigna exige MySQL. Migrar a PostgreSQL para aprovechar el plan gratuito de Render implicaría reescribir parte del esquema, por lo que se optó por mantener MySQL y documentar las alternativas de alojamiento.
Búsqueda de texto	Usa coincidencia parcial con <code>LIKE</code> , opción que la consigna acepta como mínimo. Con un volumen mucho mayor de registros, el paso siguiente sería un índice de texto completo sobre título, resumen y palabras clave.